МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Казанский национальный исследовательский технический университет им. А.Н. Туполева – КАИ»
Институт компьютерных технологий и защиты информации
Кафедра Систем Информационной Безопасности

Лабораторная работа № 4

Дисциплина: «Аппаратные средства телекоммуникационных систем»

Тема: «Функциональная декомпозиция с глобальной суммой и критическими секциями»

Работу выполнил
Студент гр. 4404
Зарипов А.Э.

Казань, 2026

1. Цель работы

Изучение функциональной декомпозиции параллельных задач в сочетании с использованием глобальной суммы и критических секций для синхронизации доступа потоков к разделяемым данным. В качестве практической задачи выбрано вычисление значения по формуле (Σx + Σy + Σz) / (3N), где N — количество точек в трёхмерном пространстве. В отличие от задачи 5, где каждая функция записывала сумму в свою отдельную переменную, в данной работе все три функции (sumX, sumY, sumZ) добавляют свои частичные суммы в одну общую глобальную переменную, защищённую критической секцией.

Задача 6 методических указаний «Параллельное программирование на OpenMP» выполняется на языке Go, поэтому конструкции OpenMP моделируются стандартными средствами языка: функциональная декомпозиция реализуется через три параллельно выполняющиеся горутины (аналог конструкции `#pragma omp parallel sections`), а критическая секция — через мьютекс `sync.Mutex` (аналог `#pragma omp critical`). sync.WaitGroup используется как неявный барьер в конце параллельной области.

Представляемые материалы: работающая программа (корректность её выполнения подтверждена скриншотами запусков, рисунки 1–2), исходный код в публичном репозитории GitHub и отчёт со скриншотами по ходу работы, ответами на поставленные вопросы и выводами.

2. Задание

Задача 6. Используя функциональную декомпозицию перепишите задачу 5 для вычисления значения по формуле: (Σx + Σy + Σz) / (3N), где N – количество точек. Используйте глобальную сумму и критические секции.

Общие требования: число потоков и размер задачи передаются программе через командную строку; запрещается использовать библиотечные функции, целиком решающие задачу; каждый запуск, о котором говорится в задаче,

подтверждается скриншотом с видимой командой запуска и полным выводом программы.

3. Ход работы

3.1. Математическая постановка задачи

Требуется вычислить значение по формуле:

result = (Σx + Σy + Σz) / (3N),

где Σx, Σy, Σz — суммы всех координат точек по соответствующим осям, N — количество точек. Исходные данные представляют собой текстовый файл, каждая строка которого содержит три числа — координаты x, y и z одной точки. В отличие от задачи 5, где вычислялись три отдельных средних (по каждой оси), в данной задаче все три суммы складываются в одну глобальную сумму, которая затем делится на 3N. Это означает, что три параллельные функции должны обновлять одну общую переменную, что требует синхронизации через критическую секцию.

3.2. Работа с входным файлом: генерация и чтение точек

Программа поддерживает отдельный режим генерации входного файла командой вида `go run . gen <task_size> [file]`. Генератор создаёт детерминированные точки с координатами (i, 2·i, 3·i) для i = 0 … N-1. Для таких точек ожидаемый результат можно вычислить аналитически:

Σx + Σy + Σz = N(N-1)/2 + N(N-1) + 3N(N-1)/2 = 3N(N-1);

result = 3N(N-1) / (3N) = N - 1.

Для N = 2 000 000 ожидаемый результат равен 1999999.

Фрагмент кода (lab4/main.go, функция generatePoints):

```
func generatePoints(filename string, n int) error {
    f, err := os.Create(filename)
    if err != nil { return err }
    defer f.Close()
    w := bufio.NewWriterSize(f, 1<<20)
    for i := 0; i < n; i++ {
```

```
            fmt.Fprintf(w, "%d %d %d\n", i, 2*i, 3*i)
        }
        return w.Flush()
    }
```

Чтение точек выполняется функцией readPoints: файл открывается через bufio.Scanner, каждая строка разбирается функцией strings.Fields, первые три токена преобразуются в float64 через strconv.ParseFloat. Пустые строки и строки, начинающиеся с «#», пропускаются.

Фрагмент кода (lab4/main.go, функция readPoints):

```
func readPoints(filename string, need int) ([]float64, []float64, []float64,
error) {
    f, err := os.Open(filename)
    if err != nil { return nil, nil, nil, err }
    defer f.Close()
    sc := bufio.NewScanner(f)
    xs := make([]float64, need)
    ys := make([]float64, need)
    zs := make([]float64, need)
    count := 0
    for sc.Scan() {
        line := strings.TrimSpace(sc.Text())
        if line == "" || strings.HasPrefix(line, "#") { continue }
        fields := strings.Fields(line)
        x, _ := strconv.ParseFloat(fields[0], 64)
        y, _ := strconv.ParseFloat(fields[1], 64)
        z, _ := strconv.ParseFloat(fields[2], 64)
        xs[count], ys[count], zs[count] = x, y, z
        count++
        if count == need { break }
    }
    if count < need { return nil, nil, nil, fmt.Errorf(...) }
    return xs, ys, zs, nil
}
```

Команда генерации входного файла и результат её работы показаны на рисунке 1. Генерация двух миллионов точек занимает порядка единиц секунд, после чего файл points.txt готов для использования в основном запуске.

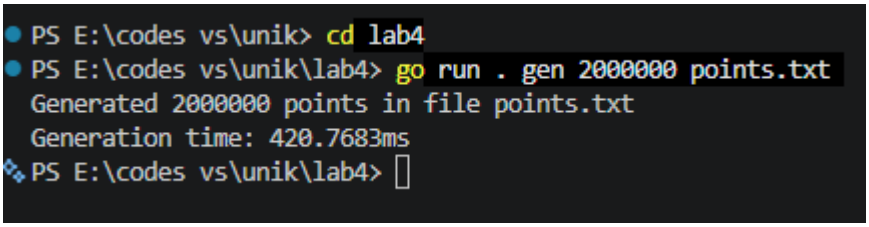


Рисунок 1 — Генерация входного файла на 2 000 000 точек

3.3. Функциональная декомпозиция с глобальной суммой и критическими секциями

Вычисление выполняется тремя параллельными горутинами (аналог `#pragma omp parallel sections`):

— функция sumX вычисляет сумму всех координат x;

— функция sumY вычисляет сумму всех координат y;

— функция sumZ вычисляет сумму всех координат z.

Ключевое отличие от задачи 5: каждая функция добавляет свою частичную сумму в одну общую глобальную переменную `globalSum`. Поскольку три горутины выполняются параллельно и одновременно пытаются обновить одну переменную, без синхронизации возникла бы гонка данных: две горутины могли бы одновременно прочитать старое значение, прибавить свои суммы и записать результат, потеряв одну из них. Для предотвращения этого обновление `globalSum` выполняется внутри критической секции, защищённой мьютексом `sync.Mutex` (аналог `#pragma omp critical` в OpenMP).

Фрагмент кода (lab4/main.go, функция computeWithCriticalSections):

```
func computeWithCriticalSections(xs, ys, zs []float64) float64 {
    var globalSum float64
    var mu sync.Mutex
    var wg sync.WaitGroup
    wg.Add(3)

    // Функция 1: сумма по x
    go func() {
        defer wg.Done()
        sx := sumSlice(xs)
        mu.Lock()          // критическая секция
        globalSum += sx
        mu.Unlock()
    }()

    // Функция 2: сумма по y
    go func() {
        defer wg.Done()
        sy := sumSlice(ys)
        mu.Lock()          // критическая секция
        globalSum += sy
        mu.Unlock()
    }()
```

```
        // Функция 3: сумма по z
        go func() {
            defer wg.Done()
            sz := sumSlice(zs)
            mu.Lock()          // критическая секция
            globalSum += sz
            mu.Unlock()
        }()

        wg.Wait()
        return globalSum
    }
```

Аналог в OpenMP:

```
    double global_sum = 0.0;
    #pragma omp parallel sections
    {
        #pragma omp section
        {
            double sx = calc_sum_x();
            #pragma omp critical
            global_sum += sx;
        }
        #pragma omp section
        {
            double sy = calc_sum_y();
            #pragma omp critical
            global_sum += sy;
        }
        #pragma omp section
        {
            double sz = calc_sum_z();
            #pragma omp critical
            global_sum += sz;
        }
    }
    double result = global_sum / (3.0 * N);
```

После завершения всех трёх горутин (барьер через `sync.WaitGroup`) глобальная сумма делится на 3N для получения итогового результата.

Примечание: функциональная декомпозиция жёстко ограничена тремя горутинами (по числу функций), поэтому увеличение числа потоков сверх 3 через `runtime.GOMAXPROCS` не даёт прироста в вычислительной части программы. Число потоков передаётся через командную строку для соответствия общим требованиям к лабораторным работам.

На рисунке 2 показан запуск программы с 8 потоками и 2 000 000 точек, прочитанных из файла points.txt. Вывод содержит имя файла, число потоков,

время чтения, время вычислений, глобальную сумму и итоговый результат, совпадающий с ожидаемым значением 1999999.

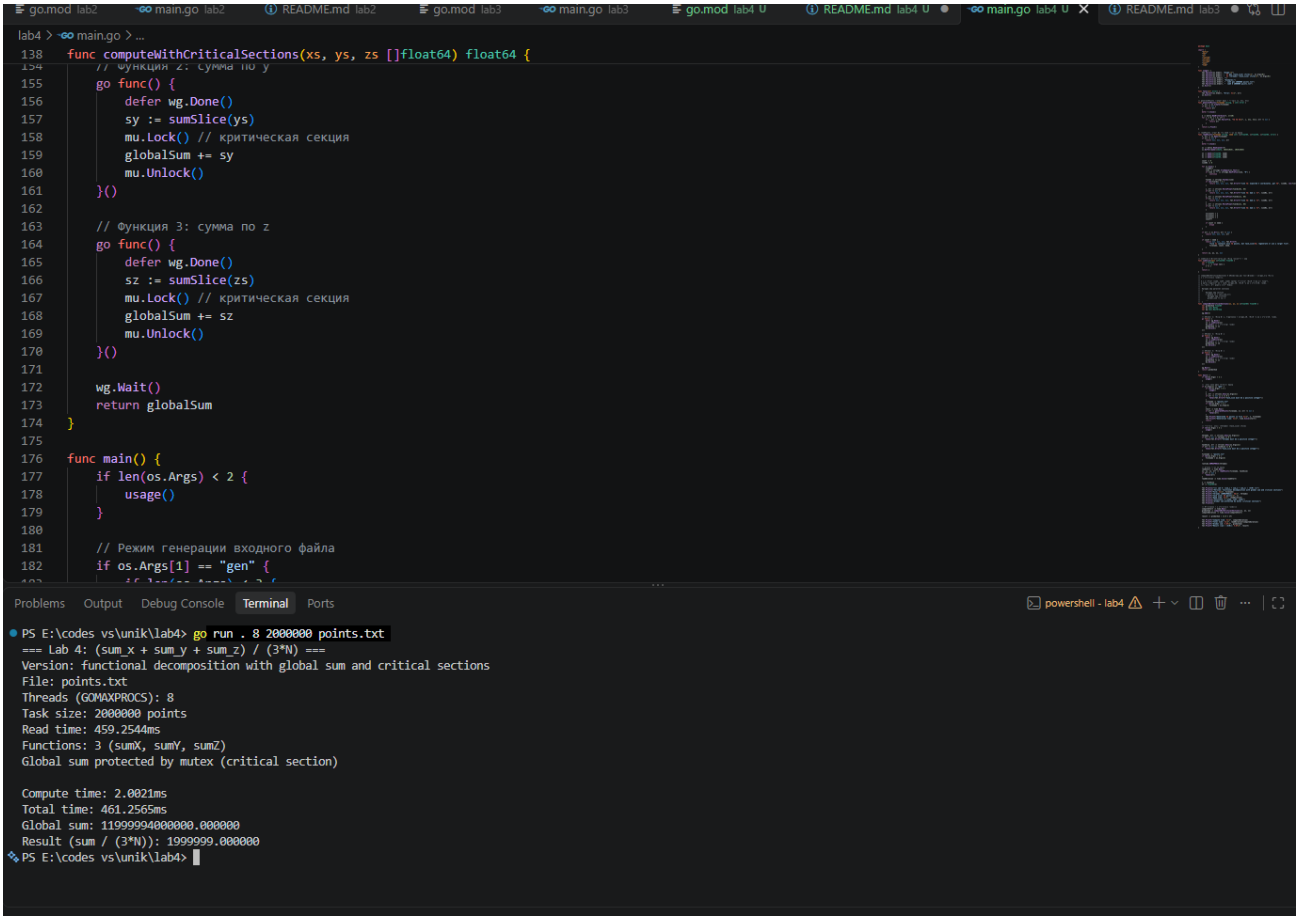


Рисунок 2 — Вычисление (Σx + Σy + Σz) / (3N) с критическими секциями

Таблица 1 — Сравнение задач 5 и 6

Параметр	**Задача 5**	**Задача 6**
Формула	три отдельных средних: Σx/N, Σy/N, Σz/N	одно общее: (Σx+Σy+Σz)/(3N)
Хранение сумм	три отдельные переменные (sx, sy, sz)	одна глобальная переменная (globalSum)
Критические секции	не требуются (нет разделяемых записей)	требуются (три горутины пишут в одну переменную)
Аналог в OpenMP	parallel sections	parallel sections + critical
Синхронизация	только барьер (WaitGroup)	барьер + мьютекс на обновление суммы

4. Ответы на вопросы, поставленные в задачах

4.1. Вопрос задачи 6: использование глобальной суммы и критических секций

Явный вопрос в задаче 6 не поставлен, однако требуется продемонстрировать использование глобальной суммы и критических секций при функциональной декомпозиции. По результатам запуска (рисунок 2):

— три горутины (sumX, sumY, sumZ) параллельно вычисляют частичные суммы и добавляют их в одну глобальную переменную через критическую секцию;

— результат (1999999 для N = 2 000 000) совпадает с аналитически ожидаемым значением, что подтверждает корректность реализации;

— без критической секции результат был бы недетерминированным из-за гонки данных при одновременном обновлении глобальной суммы тремя горутинами.

5. Выводы

1. Реализована программа вычисления значения по формуле (Σx + Σy + Σz) / (3N) с использованием функциональной декомпозиции и критических секций. Программа читает координаты точек из текстового файла; для подготовки файла встроен генератор точек (команда gen).

2. Функциональная декомпозиция реализована через три параллельные горутины (sumX, sumY, sumZ), каждая из которых вычисляет сумму по одной координате. Частичные суммы добавляются в одну глобальную переменную через критическую секцию (мьютекс), что предотвращает гонку данных.

3. Результат вычислений совпадает с аналитически ожидаемым значением (1999999 для N = 2 000 000), что подтверждает корректность реализации и правильность использования критической секции.

4. В отличие от задачи 5, где каждая функция писала в свою переменную и мьютексы не требовались, в задаче 6 все три функции обновляют одну общую переменную, что делает критическую секцию обязательной. Это демонстрирует

ключевую идею синхронизации при работе с разделяемыми данными в параллельных программах.